User & Authentication Management

Task 1 — Object Oriented Design Foundations

1. Problem Overview

You are building Uber like app, Every ride-sharing platform begins with its users.
Before anyone can request a ride or offer to drive, the system must manage user identities, credentials, profiles.

In this assignment, your team will design and implement the foundational user management subsystem that handles the complete account lifecycle from registration to deactivation.

2. Functional Requirements

I. The system must support three distinct account roles: RIDER, DRIVER, and ADMIN. A user's role is set at creation and cannot be changed at runtime. Each role maps to a subclass-specific profile object (Rider, Driver, AdminUser).

II. Every User object must carry a universally unique identifier (UUID), a timestamp of creation . These fields are set automatically and are immutable after creation

III. Password storage must never hold plaintext. The system must accept a raw password and store only a hash. A separate method must verify a raw password against the stored hash (use SHA-256 via Java's MessageDigest — no external libraries)

IV. The UserManager must support registering a new user, looking up a user by ID or email, listing all users of a specific role, and checking whether an email or phone is already registered

V. All repository operations must go through the provided UserRepository interface. The team must implement an InMemoryUserRepository that stores data in a Java Map keyed by ID.

VI. Rider, Driver, and AdminUser must each have their own repository interface and in-memory implementation, separate from UserRepository.

3. Classes to implement

a) enum UserRole

this is given for as here you don't need to think about it :

```
public enum UserRole {
  RIDER,
  DRIVER,
  ADMIN
}

// example use case

if(user.getRole() == UserRole.ADMIN) {
System.out.println("You are admin");
}
```

"Why we use Enum? We use enum because it represents a fixed set of values to choose from,it reduces bugs compared to using plain strings or integers."

b) User

Field	Type	Description
`id`	`String`	Unique identifier, set at construction.
`email`	`String`	Unique, validated format on set.
`phone`	`String`	Unique, non-null
`passwordHashed`	`String`	SHA-256 hash, never the raw password.
`fullName`	`String`	Non-null name.
`role`	`UserRole`	Immutable once set. set at construction.
`createdAt`	`Instant`	Set once at construction.

Methods	Description
`User(String fullName, String email, String phone, UserRole role)`	`constructor`
`// Getters for all fields`	
`public void setEmail(String email)`	`(no public setters for id, role, createdAt as they are set once at constructor)`
`public void setPhone(String phone)`	
`public void setFullName(String fullName)`	
`// implement Identifiable, Auditable, Authenticatable`	

c) Rider extends User

Field	Type	Description
riderId	String	Separate PK for the riders table
userId	String	FK to users.id (same as User.id).
wallet_balance	double	Amount of money a user have this will be used in trip
total_trips	Int	Total trips a user had

Methods	Description
Rider (String fullName, String email, String phone)	Constructor, it should handle creating User inside it
// implement **Identifiable**	

d) Driver extends User

Field	Type	Description
driverId	String	Separate PK for the drivers table
userId	String	FK to users.id (same as User.id).
wallet_balance	double	Amount of money a user have this will be used in trip
total_trips	Int	Total trips a user had
licenseNumber	String	Unique, non-null
ratingAverage	double	Default 0.0, range [0.0, 5.0].
isAvailable	boolean	Default false.

Methods	Description
Driver(String fullName, String email, String phone, String licenseNumber)	Constructor, it should handle creating User inside it
// implement **Identifiable**	
public void setAvailable(boolean available)	
public void recordCompletedTrip(double newRatingScore)	updates totalTrips and ratingAverag
// Getters for all fields	

e) InMemoryUserRepository implements UserRepository

Field	Type	Description
database	Map<String, User>	HashMap keyed by user ID.

f) InMemoryDriverRepository implements DriverRepository

Field	Type	Description
`database`	`Map<String, User>`	HashMap keyed by user ID.

g) InMemoryRiderRepository implements RiderRepository

Field	Type	Description
`database`	`Map<String, User>`	HashMap keyed by user ID.

h) UserManager

Field	Type	Description
userRepository	UserRepository	Injected via constructor.
riderRepository	RiderRepository	
driverRepository	DriverRepository	

Methods	Description
UserManager(UserRepository ur, RiderRepository rr, DriverRepository dr)	Constructor, it will be your interface to database which represented by “Repository” and user searching and creation You have one UserManger in the whole app make sure to do that so you don’t have multiple repositories in multiple locations
public Rider registerRider(String fullName, String email, String phone, String password)	You pass data to it handle calling Repository methods for adding to database “hash table” and validating data and assign the correct role
public Driver registerDriver(String fullName, String email, String phone, String password, String licenseNumber)	
public AdminUser registerAdmin(String fullName, String email, String phone, String password)	
public User getUserById(String id)	Return user object , its responsible for getting the correct Repository and getting the matched id in the hash table
public User getUserByEmail(String email)	
public List<Driver> getAvailableDrivers()	
public boolean isEmailRegistered(String email)	check if this email exist in your data base “Repository”
public boolean isPhoneRegistered(String phone)	check if this phone number exist/used in your data base “Repository”
public void setDriverAvailability(String driverId, boolean available)	When driver login he is activated through this and when logout he is de activated

4. Given Interfaces

```java
// Identifiable.java

public interface Identifiable {
/**
* Every Object like user or car or payment that have unique id must have getId()
* method to return its id
*/
String getId();
}
```

```java
// Auditable.java

import java.time.Instant;

public interface Auditable {

/** Timestamp when this entity was first created. Immutable. **/

Instant getCreatedAt();
}
```

```java
// UserRepository.java

import java.util.List;

public interface UserRepository {

       void save(User user); // add the user to the hash table

       User findById(String id);

       User findByEmail(String email);

       User findByPhone(String phone);

       List<User> findAll();

       List<User> findByRole(UserRole role);

       boolean existsByEmail(String email);

       boolean existsByPhone(String phone);

       void delete(String id);
}

// Note
// Use this in implementation to simulate you having a database you will add
    created users to this
private Map<String, User> database = new HashMap<>();

@Override
public User findById(String id) {
return database.get(id);
}

@Override
public void save(User user) {
database.put(user.getId(), user);
}
```

```java
// RiderRepository.java

import java.util.List;

public interface RiderRepository {

void save(Rider rider);

/** gets the rider object by its user id, Note that they are not the same each
have its id **/
Rider findById(String id);

Rider findByUserId(String userId);

List<Rider> getAll();

}
```

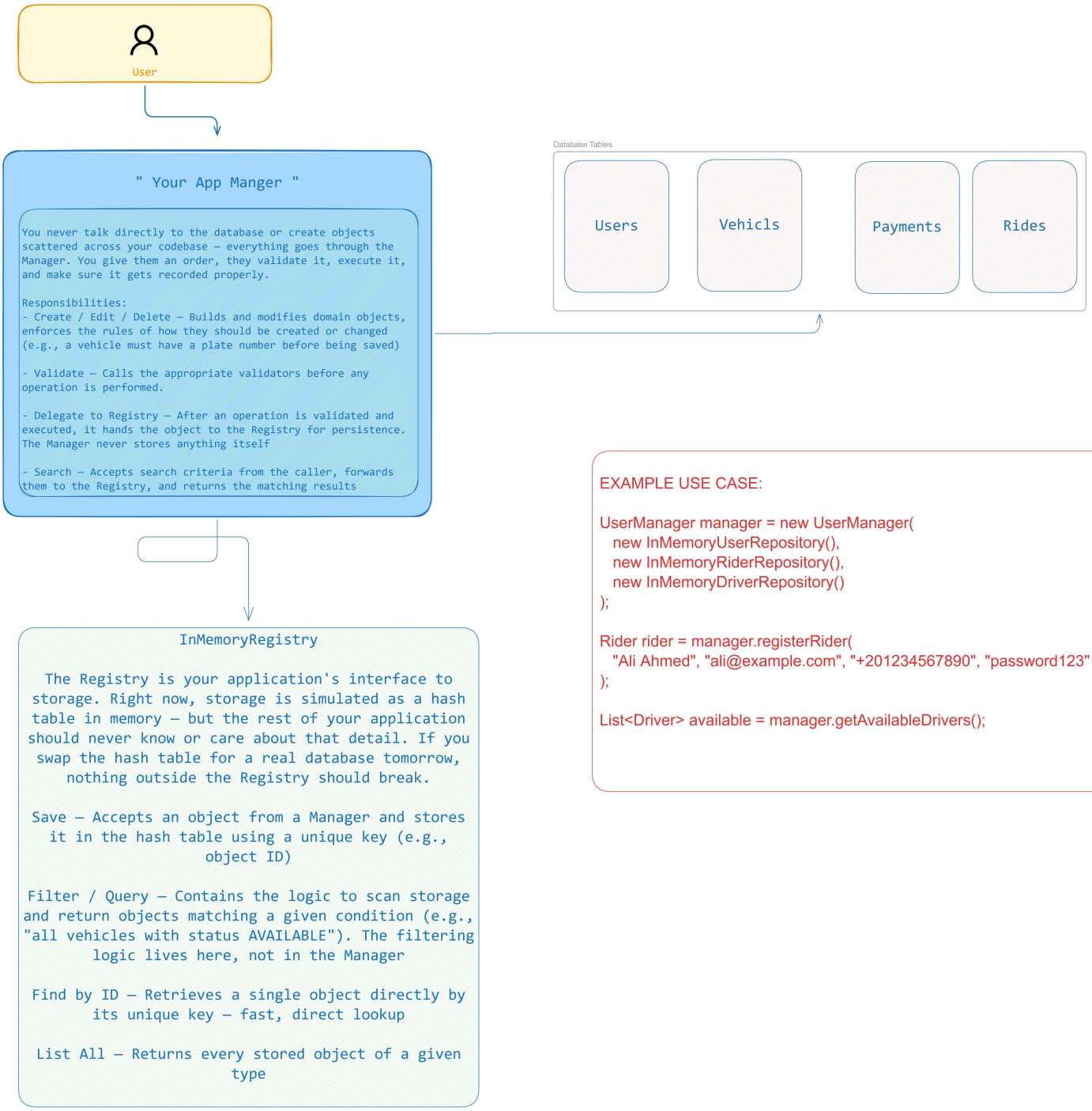
```java
// DriverRepository.java

import java.util.List;

public interface DriverRepository {
void save(Driver driver);
Driver findById(String id);
Driver findByUserId(String userId);
List<Driver> findAll();
List<Driver> findAvailable();
List<Driver> findVerified();
}
```

```java
// Authenticatable.java

import java.security.MessageDigest;
import java.nio.charset.StandardCharsets;

public interface Authenticatable {
/**
* Stores a hashed version of the given raw password in the object password field.
* Must never store plaintext.
*/
void setPassword(String rawPassword);



/**
* Returns true if the given raw password matches the stored hash.
* Must be safe against null input.
*/
boolean checkPassword(String rawPassword);
}


Example:

class User implements Authenticatable {

String password;

/**
Constructor and other logic here
**/

void setPassword(String rawPassword){
        try {
                MessageDigest md = MessageDigest.getInstance("SHA-256");
                byte[] hash = md.digest(rawPassword.getBytes(StandardCharsets.UTF_8));
                StringBuilder result = new StringBuilder();
                for (byte b : hash) {
                        result.append(String.format("%02x", b));
                }
                this.password = result.toString();

        } catch (Exception e) {
                throw new RuntimeException(e);
        }
}
```

5. Hints

- Use java.util.UUID.randomUUID().toString() to generate IDs.
- Use java.time.Instant.now() for all timestamps.
- The recordCompletedTrip method on Driver must recompute the running average correctly: newAvg = (currentAvg * totalTrips + newScore) / (totalTrips + 1).
- Instant tips & tricks (Examples):

```java
Instant now = Instant.now();
Instant oneHourLater = Instant.now().plus(1, ChronoUnit.HOURS); Instant
fiveMinutesAgo = Instant.now().minus(5, ChronoUnit.MINUTES);
fiveMinutesAgo.isBefore(oneHourLater ); // true
fiveMinutesAgo.isAfter(oneHourLater ); // false
fiveMinutesAgo.equals(oneHourLater ); // false
```



EXAMPLE USE CASE:

```
UserManager manager = new UserManager(
  new InMemoryUserRepository(),
  new InMemoryRiderRepository(),
  new InMemoryDriverRepository()
);

Rider rider = manager.registerRider(
  "Ali Ahmed", "ali@example.com", "+201234567890", "password123"
);

List<Driver> available = manager.getAvailableDrivers();
```

Repository

```
public class VehicleRepository {
private static volatile VehicleRepository instance;
public static VehicleRepository{
      if (instance == null) {
            instance = new VehicleRepository(db);
            }
      return instance;

      // rest of you constructor code here
}
}
```

Start With The Problem
Imagine you're working on a team project. There's one shared Google Doc where everyone tracks the vehicle list. Now imagine if every person on the team made their own personal copy of that document and started editing it separately.
Person A adds a vehicle. Person B doesn't see it. Person C deletes something Person A just added. Nobody has the real picture. Everything is out of sync.
This is exactly what happens in code if every part of your app creates its own VehicleRepository object. Each one has its own separate copy of the data in memory, and they never talk to each other.

The Singleton pattern solves this with one simple rule:

> **"Only one instance of this object is allowed to exist. Ever."**

The pattern has a fancy name, but the idea is just: one object, shared everywhere, created once.